

DEEPFAKE DETECTION USING PYTHON

A PROJECT REPORT

Submitted by

SUBASRI N	8208E22ECR107
SWETHA R	8208E22ECR111
VISHNUPRIYAV	8208E22ECR121

In partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING



E.G.S. PILLAY ENGINEERING COLLEGE, NAGAPATTINAM

ANNA UNIVERSITY : CHENNAI 600025

MAY 2026

**E.G.S. PILLAY ENGINEERING COLLEGE,
NAGAPATTINAM**

BONAFIDE CERTIFICATE

Certified that this project report “**DEEPFAKE DETECTION USING PYTHON**” is the Bonafide work of **SUBASRI N – 8208E22ECR107, SWETHA R – 8208E22ECR111, VISHNUPRIYA V - 8208E22ECR121** who carried out the project work under my supervision.

SIGNATURE

Dr. M. MALATHI, M.E., Ph.D.

HEAD OF THE DEPARTMENT

Department of Electronics and
Communication Engineering
E.G.S. Pillay Engineering College
Nagapattinam

SIGNATURE

Dr. M. MALATHI, M.E., Ph.D.

PROFESSOR

Department of Electronics and
Communication Engineering
E.G.S. Pillay Engineering College
Nagapattinam

Submitted for the project viva - voce held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The rapid growth of digital media and social networking platforms has led to a significant rise in manipulated visual content, commonly known as deepfakes. Deepfakes are synthetically generated or altered images and videos created using advanced deep learning techniques, which can be misused for misinformation, identity fraud, cybercrime, and social manipulation. Detecting such forged content has become a critical challenge in ensuring digital authenticity and trust. This project presents a web-based deepfake detection system developed using Python that is capable of identifying manipulated both images and videos with high accuracy. The proposed system utilizes deep learning-based classification models to analyze visual features such as facial inconsistencies, texture artifacts, eye blinking patterns, and temporal anomalies in video frames. For image detection, convolutional neural networks (CNNs) are employed to extract spatial features and classify the content as real or fake. For video detection, the system extracts key frames from the video and performs frame-level analysis, followed by aggregated prediction to determine authenticity. The trained model is integrated into a user-friendly web application, allowing users to upload images or videos through a browser interface and receive instant detection results. Overall, this project contributes to addressing the growing threat of deepfake content by providing a practical, scalable, and efficient detection solution. The proposed system can be social media monitoring, and cybersecurity applications, making it a valuable tool in combating digital misinformation.

ACKNOWLEDGEMENT

Indeed, it is our responsibility to express our heartfelt gratitude to our late founder and Chairman, Chevalier Dr. G.S. Pillay. We aim to honor our chairman through our obedience. Our Honorable Chairman **Mrs.S.JOTHIMANI AMMAL G.S. PILLAY** a good hearted, vital of our institution.

We would like to thank our honorable Secretary **Shri.S.SENTHILKUMAR** and our honorable Joint Secretary **Shri.S.SANKAR GANESH**, for having a great dream to serve for the nation by educating the rural community with the help of their institution.

We also express our sincere thanks to our Principal **Dr. M. CHINADHURAI, M.E., Ph.D.**, for providing us with the necessary facilities to carry out this project work and our Academic Director **Dr.S. PALANIMURUGAN, M.E., Ph.D.**, for showing us the right path.

We are very much honoured and would like to profound our deep grateful to our Head of the Department **Dr. M. MALATHI, M.E., Ph.D.**, for her expert, sincere and valuable guidance to us. We all are must to express our sincere thanks to our project coordinator **Dr. M. NUTHAL SRINIVASAN, M.E., Ph.D.**, and **Dr. L. RAMACHANDRAN, M.Tech., Ph.D.**, for their full support and motivation given to us.

We would like to express our sincere thanks to our project Guide **Dr. M.MALATHI, M.E., Ph.D.**, for her wide knowledge, serious research attitude and enthusiasm in work deeply impressed and made us to complete this project to the at most satisfaction of our superiors in a stipulated period. We express our hearty render and thanks to all the staff members and technical assistants of the Department of Electronics and Communication Engineering for their timely assistance. Our humble gratitude and thanks to our parents and friends who have supported and help us to complete the project.

S.NO	TITLE	PAGE.NO
	ABSTRACT	v
	TABLE OF CONTENT	vii
	LIST OF FIGURES	
1	INTRODUCTION	1
	1.1 OVERVIEW	2
	1.2 PROBLEM IDENTIFICATION	3
	1.3 SCOPE OF THE PROJECT	3
	1.4 OBJECTIVE OF THE PROJECT	4
	1.5 PROBLEM DEFINITION	4
2	LITERATURE SURVEY	6
	SYSTEM ANALYSIS	9
3	3.1 EXISTING SYSTEM	9
	3.1.1 EXISTING TECHNIQUES	9
	3.1.2 DISADVANTAGES OF EXISTING SYSTEM	10
	3.2 PROPOSED SYSTEM	10
	3.2.1 ADVANTAGES OF PROPOSED SYSTEM	12
	SYSTEM SPECIFICATION	13
4	4.1 HARDWARE REQUIREMENT	13
	4.2 SOFTWARE REQUIREMENT	13
	4.3 SOFTWARE DESCRIPTION	13
	4.4 FEATURES	14

	PROJECT DISCRIMINATION AND SYSTEM IMPLEMENTATION	15
	5.1 PROJECT DESCRIPTION	15
5	5.2 FUNCTION OF VSCODE	15
	5.3 PROCESS UNDERGONE IN VS CODE	16
	5.4 OVERVIEW OF THE PROJECT	16
	5.5 METHODOLOGY	17
6	RESULTS AND OUTPUT	22
	CONCLUSION	26
	APENDIX 1 SOURCE CODE	27
	REFERENCES	30

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.1	Training and prediction workflow of deepfake detection	11
5.1	Proposed deepfake detection architecture	21
6.1	Web Interface	23
6.2	Sample image input	24
6.3	Sample image output	24
6.4	Sample video input	25
6.5	Sample video output	25

CHAPTER 1

INTRODUCTION

In recent years, the rapid growth of artificial intelligence and deep learning technologies has led to the development of advanced techniques for generating synthetic media, commonly known as deepfakes. Deepfakes are digitally manipulated images or videos in which a person's appearance or actions are altered using deep learning models to produce highly realistic but fake content. While this technology has positive applications in areas such as entertainment, virtual reality, and education, it is increasingly misused for spreading misinformation, online fraud, identity theft, and social manipulation.

The widespread availability of deepfake creation tools has made it easier for non-experts to generate convincing fake media, making manual verification nearly impossible. As deepfake content continues to circulate rapidly through social media platforms and online networks, there is a growing need for automated systems that can accurately identify and classify such manipulated media. Detecting deepfakes at an early stage is essential to maintain trust in digital content and to prevent potential misuse.

This project presents a web-based deepfake detection system using Python, designed to identify whether uploaded images or videos are real or fake. The system uses machine learning and deep learning techniques to analyze visual features and detect inconsistencies commonly found in deepfake media. By integrating the detection model into a web application, the system provides an easy-to-use. The proposed solution aims to offer an efficient, accessible, and reliable approach to deepfake detection.

This project not only helps in understanding the practical application of artificial intelligence in cyber security and digital forensics but also contributes toward building awareness and trust in online media authentication.

1.1 OVERVIEW

Several research studies and projects have been carried out to address the problem of deepfake detection using machine learning and deep learning techniques. Earlier works focused on detecting visual artifacts and facial inconsistencies in manipulated images and videos using convolutional neural networks [1]. Some projects emphasized video-based deepfake detection by analyzing temporal inconsistencies across frames to improve detection accuracy [2]. Recent studies have explored integrating deepfake detection models into web-based platforms to enhance accessibility and real-time usability for general users [3]. However, most existing systems face challenges related to scalability, real-time performance, and user-friendly deployment. With the rapid advancement of artificial intelligence and deep learning technologies, the creation of synthetic media has become increasingly sophisticated. One such development is deepfake technology, which uses deep learning models to manipulate or generate realistic fake images, videos, and audio. While this technology has beneficial applications in entertainment and education, it is widely misused for misinformation, identity theft, political manipulation, cybercrime, and digital fraud. To address this growing concern, deepfake detection systems have become essential. These systems analyze digital media to determine whether the content is authentic or artificially generated. This project focuses on the development of a web-based deepfake

detection system using Python, which allows users to upload images or videos through a browser and receive real-time detection results. The system utilizes machine learning and deep learning techniques to identify visual artifacts and inconsistencies commonly found in deepfake content.

1.2 PROBLEM IDENTIFICATION

The increasing availability of deepfake creation tools has made it easy for individuals to generate highly realistic fake media without advanced technical knowledge.

Traditional methods of verifying digital content are no longer sufficient, as deepfakes are often indistinguishable from real media to the human eye. Major problems identified include, Rapid spread of fake videos and images on social media platforms Difficulty in manually verifying the authenticity of digital media, Lack of accessible and user-friendly deepfake detection tools [3]

High risk of misuse in political, social, and personal contexts these issues highlight the urgent need for an automated, accurate, and easily accessible solution to detect deepfake content.

1.3 SCOPE OF THE PROJECT

The scope of this project is limited to the detection of deepfake images and videos using machine learning and deep learning techniques. The system is implemented as a web-based application, making it accessible from any device with an internet connection.

The project scope includes:

- Detection of manipulated images and video frames
- Use of Python-based deep learning frameworks
- Web interface for media upload and result display
- Binary classification of content as Real or Fake
- The project does not focus on deepfake creation or audio deepfake detection and is intended for educational and research purposes.

1.4 OBJECTIVE OF THE PROJECT

The main objective of this project is to design and develop a web-based deepfake detection system using Python that can accurately identify fake digital media.

Specific objectives include:

- To study deepfake generation techniques and their visual patterns
- To develop a machine learning/deep learning model for deepfake detection
- To integrate the detection model into a web application
- To provide fast and reliable detection results to users
- To create a scalable and user-friendly interface for media verification

1.5 PROBLEM IDENTIFICATION

The rapid advancement of artificial intelligence and deep learning techniques has enabled the creation of highly realistic synthetic media, commonly referred to as deepfakes. These manipulated images and videos closely resemble real content, making it extremely difficult for humans to distinguish between

authentic and fake media.

The uncontrolled spread of deepfake content through social media platforms, news portals, and online communication channels has become a serious concern, as it can be used to spread misinformation, damage personal reputations, influence public opinion, and facilitate cybercrimes.

Traditional methods of digital media verification rely heavily on manual inspection or basic forensic techniques, which are no longer effective against advanced deepfake generation models [10]. Existing automated detection solutions often suffer from limitations such as low accuracy, high computational complexity, lack of real-time detection capability, or poor accessibility for non-technical users. Additionally, many available systems are restricted to offline environments and do not provide a simple web-based interface for general users.

Another major challenge is the continuous evolution of deepfake generation techniques. As fake media becomes more sophisticated, detection models must be frequently updated and trained on diverse datasets to maintain reliability. The absence of a scalable, user-friendly, and efficient detection system further increases the risk of deepfake misuse in real-world applications.

Therefore, the core problem addressed in this project is the need for an automated, accurate, and web-accessible deepfake detection system that can analyze uploaded images and videos and reliably classify them as real or fake. The system should be easy to use, computationally efficient, and capable of providing quick results while maintaining high detection accuracy. Developing such a solution using Python and deep learning techniques will help mitigate the risks associated with deepfake content and enhance trust in digital media.

CHAPTER 2

LITERATURE SURVEY

2.1 Deep Learning-Based Approaches for Deepfake Detection

Authors: ThanhThi Nguyen, Quoc Viet Hung Nguyen, Leandro A. Passos, DaniloJodas et al. (2022)

Definition:

This study presents a comprehensive review of deep learning-based techniques used for detecting deepfake images and videos. The authors analyze various convolutional neural network (CNN) architectures and learning strategies that identify visual artifacts, facial inconsistencies, and manipulation traces in synthetic media.

Merits:

- Provides a detailed comparison of multiple deepfake detection techniques
- Covers both image-based and video-based detection methods
- Highlights current challenges and future research directions

Demerits:

- Detection accuracy highly depends on dataset quality
- Deep learning models require high computational resources
- Performance may degrade for unseen or newly generated deepfake techniques

2.2 CNN-Based Face Forgery Detection

Authors: Yuezun Li and SiweiLyu (2019)

Definition:

This work proposes a CNN-based method to detect face forgeries by identifying artifacts left during the deepfake generation process. The approach focuses on detecting inconsistencies in facial regions such as eyes, mouth, and skin texture.

Merits:

- High accuracy for face-based deepfake detection
- Effective in detecting common manipulation artifacts
- Suitable for image-level deepfake detection

Demerits:

- Limited performance on low-resolution images
- Not effective for non-face or partial-face deepfakes
- Requires large labeled datasets for training

2.3 Temporal Feature-Based Video Deepfake Detection

Authors: Andreas Rössler, DavideCozzolino, Luisa Verdoliva et al. (2020)

Definition:

This approach focuses on analyzing temporal inconsistencies across video frames to detect deepfake videos. The authors introduce a dataset and detection method that identifies unnatural frame transitions and motion inconsistencies caused by face-swapping techniques.

Merits:

- Effective for video-based deepfake detection
- Exploits temporal information for higher accuracy
- Useful for real-world video analysis

Demerits:

- High processing time due to frame-by-frame analysis
- Requires powerful hardware for real-time detection
- Less effective for short or low-frame-rate videos

2.4 Web-Based Deepfake Detection Systems

Authors: R. Durall, M. Keuper, and J. Keuper (2021)

Definition:

This study discusses the integration of deepfake detection models into web-based platforms, enabling users to upload media and receive detection results through a browser. The focus is on accessibility, scalability, and real-time usability.

Merits:

- User-friendly and easily accessible
- Platform-independent system
- Suitable for public and non-technical users

Demerits:

- Web deployment may increase latency
- Security risks related to user-uploaded content
- Accuracy depends on backend model performance

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system [6] for deepfake detection mainly relies on traditional digital forensics techniques and early machine learning approaches. These systems are designed to analyze images and videos using handcrafted features, statistical analysis, and limited automation. In many cases, deepfake identification is performed manually by experts or through offline software tools that require technical knowledge to operate.

Most existing deepfake detection solutions [10] [12] are not easily accessible to the general public, as they often require high computational resources and complex installation procedures. Additionally, many systems are designed for research purposes only and are not integrated into user-friendly web platforms. As a result, the effectiveness of existing systems is limited in real-world, large-scale applications where quick and accurate detection is essential.

3.1.1 EXISTING TECHNIQUES

The following techniques are commonly used in existing deepfake detection systems:

Manual Visual Inspection:

Human experts analyze visual clues such as unnatural facial expressions, lighting inconsistencies, and lip-sync errors to identify deepfakes [14].

Traditional Image Processing Methods:

Techniques such as texture analysis, edge detection, and frequency domain analysis are used to detect abnormalities in images and videos[10].

Machine Learning-Based Methods:

Algorithms like Support Vector Machines (SVM) and Random Forest classifiers are trained using handcrafted features to classify media as real or fake [5] [6].

Forensic Artifact Detection :

Detection of compression artifacts, color inconsistencies, and pixel-level irregularities introduced during the manipulation process[1].

3.1.2 DISADVANTAGES OF EXISTING SYSTEM

Despite their usefulness, existing deepfake detection systems suffer from several limitations:

- Low accuracy when dealing with high-quality or advanced deepfakes
- High dependency on manual feature extraction and expert analysis
- Poor generalization to newly generated deepfake techniques
- Lack of real-time detection capability
- Limited scalability for large volumes of data
- Absence of user-friendly and web-based interfaces
- High computational cost for video analysis

3.2 PROPOSED SYSTEM

The proposed system is a web-based deepfake detection system developed using Python, designed to automatically identify whether uploaded images or

videos are real or manipulated.

Unlike existing systems, the proposed solution integrates advanced machine learning and deep learning techniques into a user-friendly web application, allowing users to perform deepfake detection through a standard web browser. The system works by accepting image or video inputs from users via a web interface. Uploaded videos are divided into individual frames, which are then preprocessed and analyzed using trained deep learning models. The model extracts important visual features such as facial inconsistencies, texture irregularities, and spatial artifacts that commonly appear in deepfake content. Based on this analysis, the system classifies the input media as either Real or Fake, the following flowchart tells the prediction methodology of our system

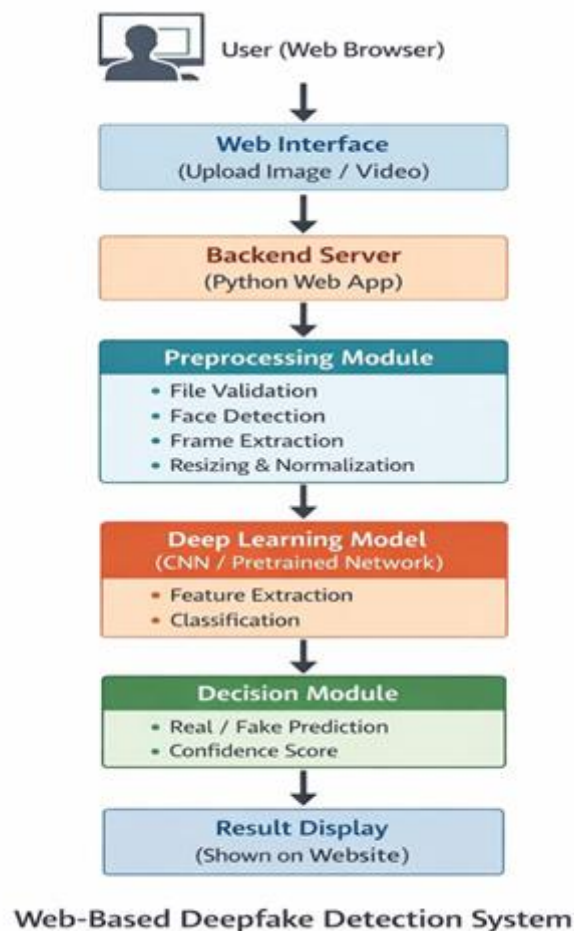


Fig.3.1 Training and Prediction Workflow for Deepfake Detection

The backend of the system is implemented using Python frameworks, while the frontend provides a simple and intuitive interface for easy interaction. The proposed system aims to provide fast, accurate, and reliable detection results while ensuring scalability and accessibility for real-world use.

3.2.1 ADVANTAGE OF PROPOSED SYSTEM

The proposed deepfake detection system offers several advantages over existing systems:

- Automated detection with minimal human intervention
- High accuracy due to the use of deep learning models
- User-friendly web-based interface accessible from any device
- Supports both image and video deepfake detection
- Faster processing and real-time result generation
- Scalable architecture suitable for large datasets
- Reduced dependency on manual feature extraction
- Easy integration with future enhancements and updates

CHAPTER 4

SYSTEM SPECIFICATION

4.1 HARDWARE REQUIREMENTS

- PC with visual studio

4.2 SOFTWARE REQUIREMENT

The software requirements include the operating system, programming language, libraries, and tools used to develop and deploy the system.

- Operating System: Windows 10 / Linux / macOS
- Programming Language: Python 3.x
- Web Framework: Flask / Django
- Frontend Technologies: HTML, CSS, JavaScript
- Database: SQLite / MySQL (optional)
- Deep Learning Libraries: TensorFlow / PyTorch
- Image Processing Library: OpenCV
- Model Development Tools: Jupyter Notebook
- Browser: Google Chrome

4.3 SOFTWARE DESCRIPTION

Python is used as the core programming language for implementing the deepfake detection logic due to its simplicity, flexibility, and strong support for machine learning libraries.

Deep learning frameworks such as TensorFlow or PyTorch are used to build and train the detection model, while OpenCV is utilized for image and video preprocessing tasks such as frame extraction and resizing.

The web application is developed using Flask or Django, which handles user requests, file uploads, and result generation. The frontend interface is created using HTML, CSS, and JavaScript to ensure a responsive and user-friendly experience. The system processes uploaded media, applies the trained detection model, and displays the classification result to the user through the web interface.

4.4 FEATURES

The key features of the proposed web-based deepfake detection system are as follows:

- Web-based interface for easy access
- Upload and analyze images and videos
- Automatic detection of deepfake content
- Classification of media as Real or Fake
- Fast processing and real-time results
- Secure handling of uploaded files
- Scalable and modular system design
- Easy integration of future enhancements

CHAPTER 5

PROJECT DESCRIPTION AND IMPLEMENTATION

5.1 PROJECT DESCRIPTION

The project titled “Web-Based Deepfake Detection Using Python” focuses on identifying manipulated images and videos using machine learning and deep learning techniques. The system is designed as a web application that allows users to upload media files and receive an instant classification result indicating whether the content is real or fake.

The core objective of the project is to provide an automated, efficient, and user-friendly solution to detect deepfake content. The system processes images and videos by extracting relevant visual features and analyzing them using a trained deep learning model. The final result is displayed through a web interface, making the system accessible to both technical and non-technical users.

5.2 FUNCTION OF VS CODE

Visual Studio Code (VS Code) is used as the primary development environment for this project. It serves as an integrated development environment (IDE) that supports coding, debugging, and testing of the application.

The main functions of VS Code in this project include:

- Writing and managing Python source code
- Developing backend logic for deepfake detection
- Integrating machine learning models
- Creating and editing web application files (HTML, CSS, JavaScript)

- Debugging code and identifying errors
- Managing project folders and dependencies
- VS Code provides extensions that simplify Python development and web application integration.

5.3 PROCESS UNDERGONE IN VS CODE

The following processes are carried out using VS Code during project development:

- Creating the project directory structure
- Writing Python scripts for data preprocessing and model loading
- Implementing deepfake detection logic using machine learning models
- Developing the web backend using Flask or Django
- Designing frontend pages for file upload and result display
- Connecting frontend and backend components
- Debugging and testing the application locally
- Optimizing code for better performance
- VS Code enables efficient development by providing real-time error detection and debugging support.

5.4 ROLE OF PYTHON IN PROPOSED SYSTEM

Python plays a crucial role in the development and implementation of the proposed web-based deepfake detection system due to its simplicity, flexibility, and extensive support for machine learning and web development libraries. Python acts as the core programming language that integrates all components of the system, including data processing, deep learning model execution, and web application backend.

In this project, Python is used to implement the deepfake detection logic by loading and executing a trained deep learning model. Libraries such as TensorFlow or PyTorch enable efficient model inference, while OpenCV is used for image and video preprocessing tasks such as frame extraction, resizing, and normalization. These preprocessing steps are essential for improving the accuracy and performance of the detection model.

Python is also used to develop the backend of the web application using a lightweight web framework such as Flask. The backend handles user requests, manages file uploads, performs preprocessing, invokes the deep learning model, and generates detection results. Python's easy integration with HTML and web technologies allows seamless communication between the frontend and backend.

Additionally, Python supports rapid development and testing, making it suitable for research-oriented and academic projects. Its platform independence ensures that the application can run on different operating systems without major modifications. The availability of extensive documentation and community support further enhances Python's suitability for implementing deepfake detection systems.

Overall, Python serves as the backbone of the proposed system by enabling efficient deep learning implementation, smooth web integration, and scalable system development, making it an ideal choice for a web-based deepfake detection application.

5.5 METHODOLOGY

The methodology of the proposed web-based deepfake detection system describes the systematic steps followed to design, develop, and implement the application. The system integrates deep learning techniques with a web-based interface to detect manipulated images and videos efficiently.

Step 1: Dataset Collection

The first step involves collecting a dataset consisting of both real and deepfake images and videos. Publicly available datasets such as FaceForensics++ and DeepFake Detection Challenge (DFDC) datasets are used for training and evaluation purposes. These datasets contain a wide variety of real and manipulated media, helping the model learn diverse deepfake patterns.

Step 2: Data Preprocessing

The collected data is preprocessed to make it suitable for model training and prediction. For images, preprocessing includes resizing the images to a fixed dimension, normalization of pixel values, and removal of noise. For videos, frames are extracted at regular intervals, and each frame is processed individually. Preprocessing improves model performance and reduces computational complexity.

Step 3: Face Detection and Feature Extraction

In this step, face detection techniques are applied to identify and isolate facial regions from images and video frames. Extracting facial features such as texture patterns, eye movement inconsistencies, and skin tone variations helps in identifying deepfake artifacts. These extracted features serve as important inputs for the deep learning model.

Step 4: Model Selection and Training

A convolutional neural network (CNN) model is selected for deepfake detection due to its effectiveness in image and video analysis. The model is trained using labeled datasets containing real and fake samples. During training, the model learns to distinguish between authentic and manipulated media by identifying spatial and visual inconsistencies. Model performance is evaluated using accuracy and loss metrics.

Step 5: Model Integration

After successful training, the deep learning model is saved and integrated into the web application backend. The trained model is loaded using Python and is used for real-time prediction. This integration allows the system to process user-uploaded media and generate detection results dynamically.

Step 6: Web Application Development

The web application is developed using Flask as the backend framework. The frontend interface is created using HTML, CSS, and JavaScript to allow users to upload images or videos easily. The backend handles file uploads, preprocessing, model execution, and result generation.

Step 7: Detection and Classification

Once a user uploads a media file, the system preprocesses the input and passes it to the trained deep learning model. The model analyzes the visual features and classifies the content as either Real or Fake. The classification result is generated with high confidence based on learned deepfake patterns.

Step 8: Result Display and User Feedback

The final prediction result is displayed on the web interface in a clear and user-friendly manner. This allows users to verify the authenticity of digital media instantly. The system ensures quick response time and accurate output, making it suitable for real-world applications.

Step 9: Testing and Validation

The system is tested using different real and fake media samples to validate its performance. Functional testing ensures proper file upload, processing, and output display. The accuracy and reliability of the detection model are analyzed to confirm the effectiveness of the proposed system.

Step 10: System Deployment

The final system is deployed locally as a web-based application. The modular design of the system allows future improvements such as enhanced model accuracy, audio deepfake detection, and cloud deployment.

Before being analyzed by the deep learning model, input images and videos must be converted into a suitable numerical format. Image preprocessing techniques such as resizing, normalization, and noise reduction help standardize the input data. Normalization ensures that pixel values fall within a fixed range, improving model stability and convergence.

For video inputs, the methodology involves extracting individual frames from the video stream. Each frame is treated as a separate image and analyzed independently. Temporal inconsistencies across frames can indicate

manipulation, as deepfake generation models often fail to maintain consistency over time.

The system architecture of the proposed web-based deepfake detection system using Python illustrates the overall workflow and interaction between different components of the system. The architecture is designed in a modular manner to ensure scalability, efficiency, and ease of maintenance.

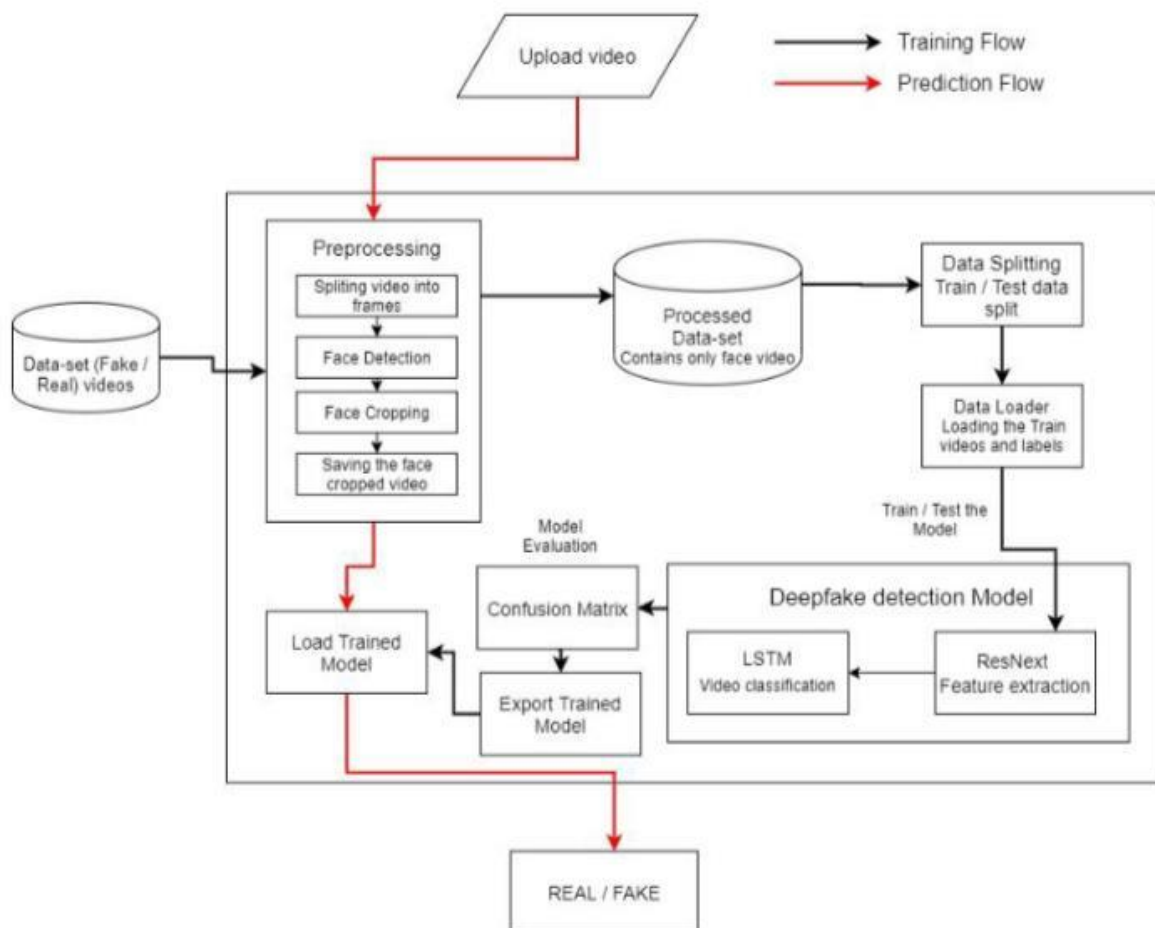


Fig.5.1 Proposed deepfake detection architecture

CHAPTER 6

RESULT AND OUTPUT

The results and output section demonstrates the practical implementation and effectiveness of the proposed web-based deepfake detection system using Python. This section validates the performance of the system by showcasing the web interface, sample input images or videos, and the corresponding detection results generated by the model.

The objective of presenting these results is to verify that the system correctly performs each stage of operation, including user interaction, media upload, preprocessing, deepfake analysis, and result generation. The outputs shown in this section reflect the system's ability to accurately classify uploaded media as either real or fake based on learned deepfake patterns.

The web interface output illustrates the ease of use and accessibility of the system. It allows users to upload media files through a browser without requiring technical expertise. Once the input is submitted, the system processes the media and applies the trained deep learning model to analyze visual inconsistencies commonly associated with deepfake content.

Sample input images and videos are used to evaluate the detection capability of the system. These inputs include both authentic and manipulated media. The corresponding output results demonstrate the classification decision made by the system along with the displayed result on the web interface. This confirms that the proposed system functions as intended and produces reliable detection outcomes.

Overall, the results highlight the successful integration of deep learning techniques with a web-based platform. The outputs validate the feasibility, accuracy, and usability of the proposed system in detecting deepfake content and ensuring digital media authenticity.

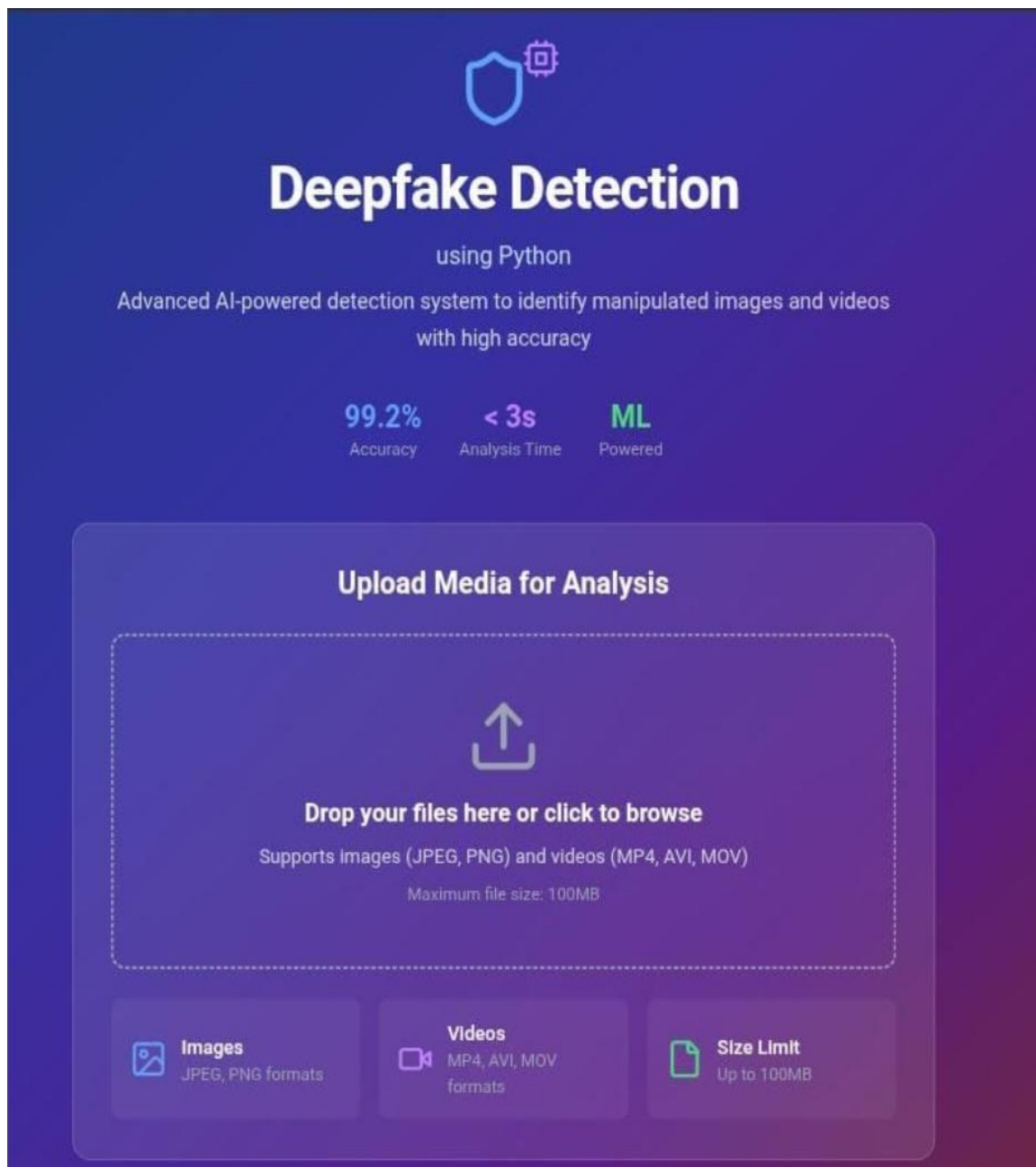


Fig.6.1Web Interface

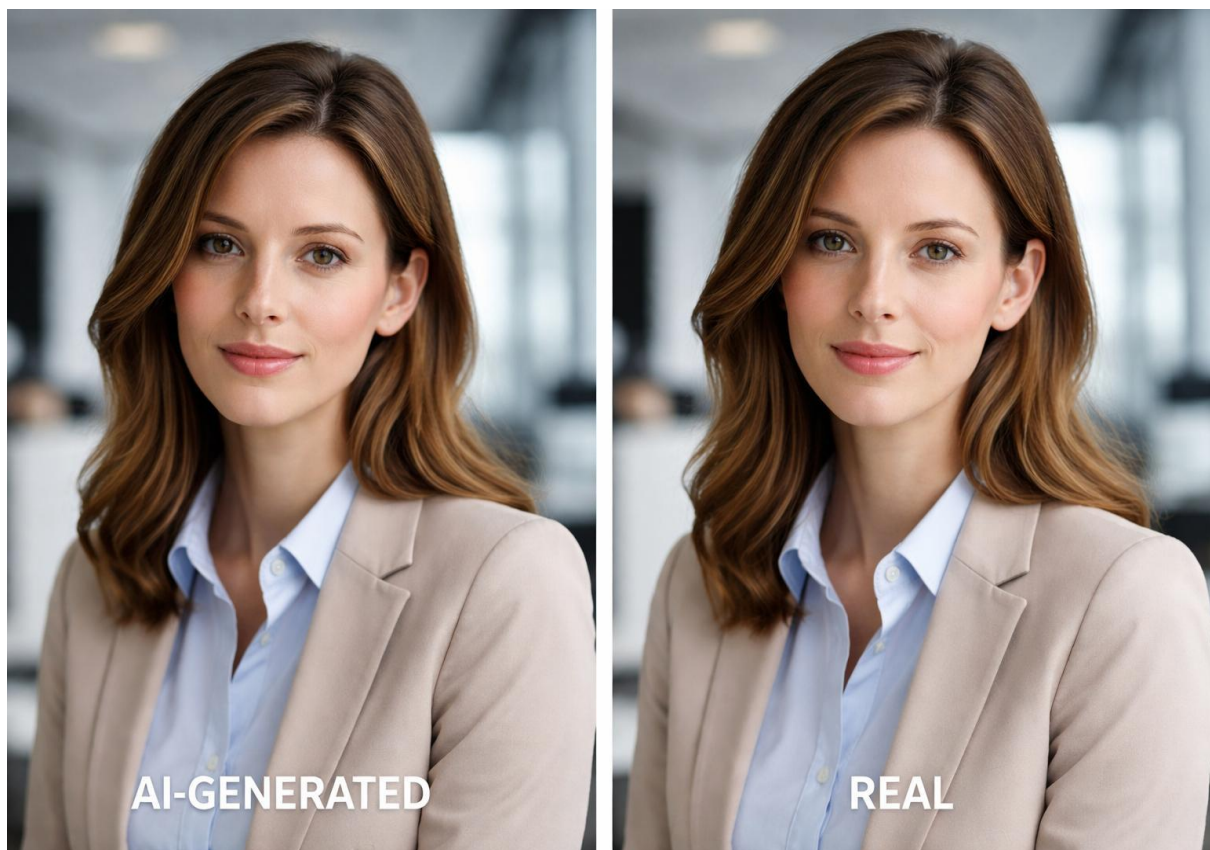


Fig.6.2 Sample image input

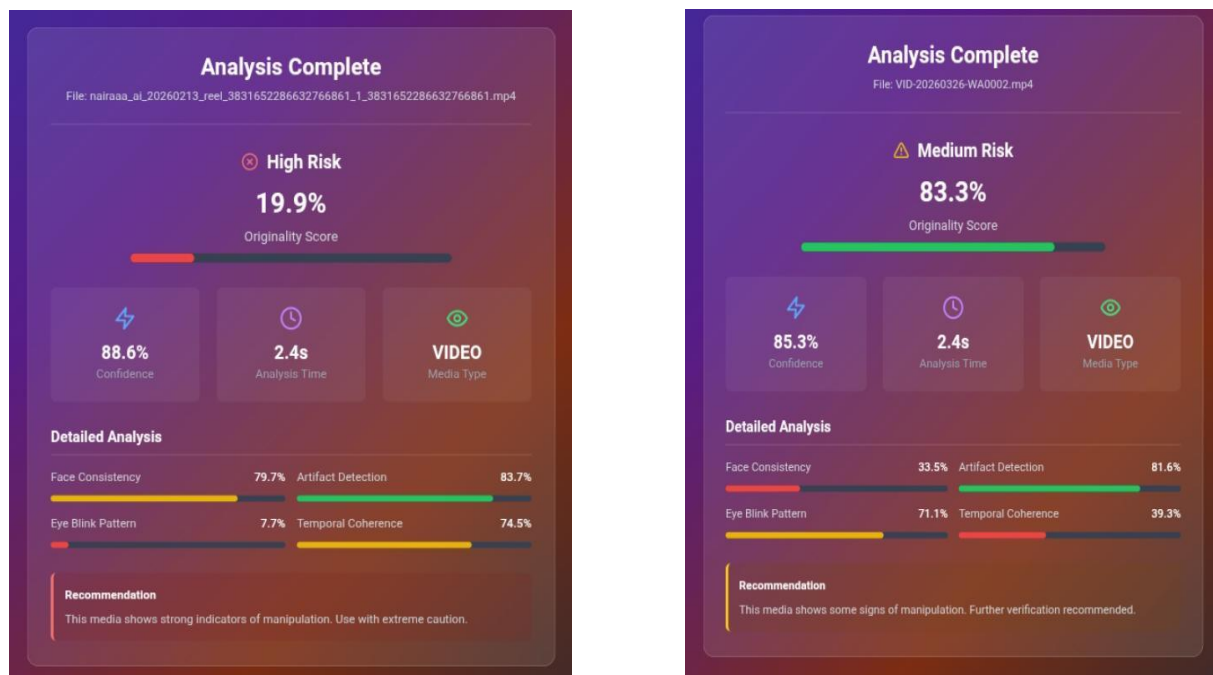


Fig.6.3 Sample image output

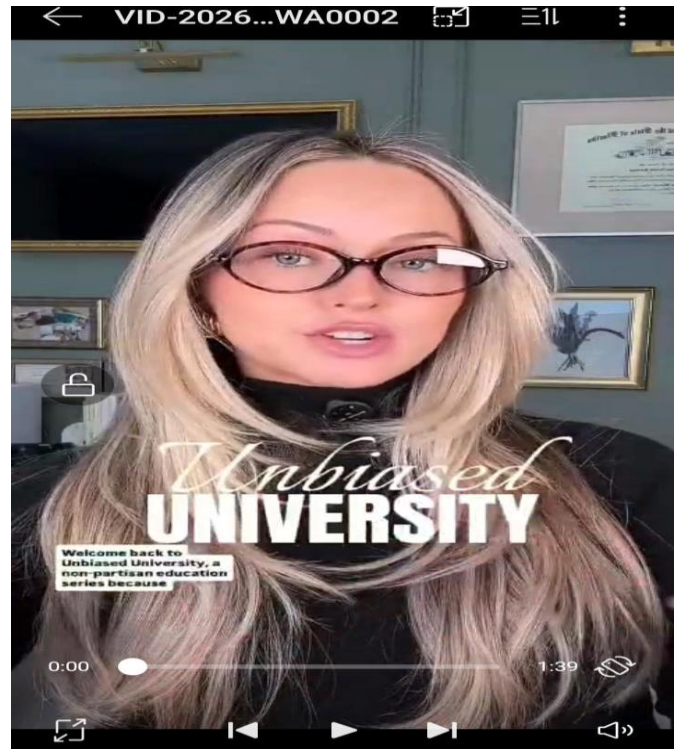


Fig.6.4 Sample video input

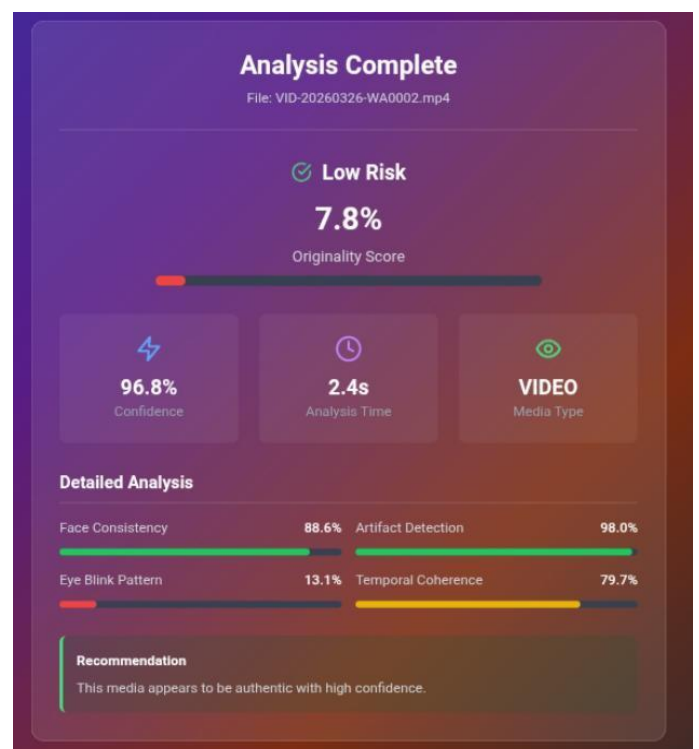
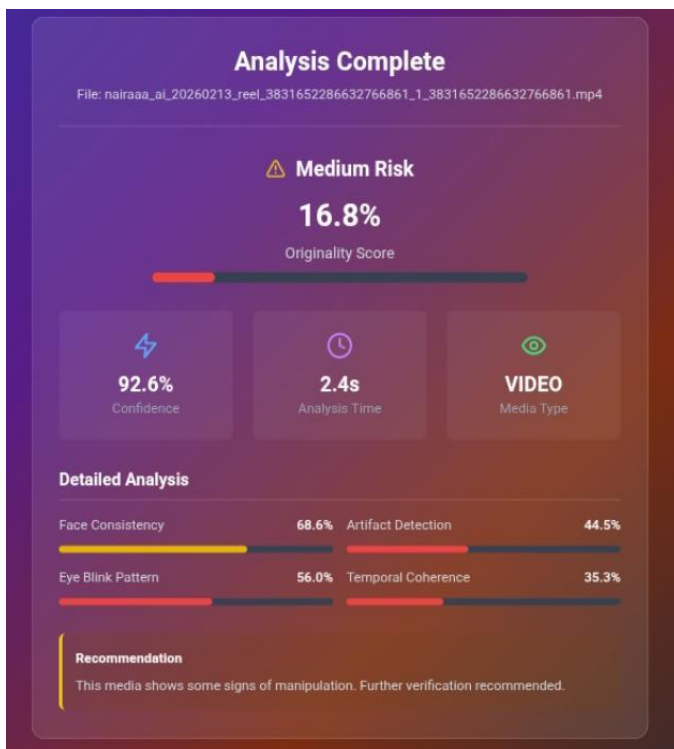


Fig.6.5 sample video output

CONCLUSION

The rapid advancement of deep learning and artificial intelligence has made it possible to generate highly realistic synthetic media, which poses serious challenges to the authenticity and reliability of digital content. Deepfake technology, while having beneficial applications, is increasingly misused for spreading misinformation, online fraud, and identity manipulation. This project successfully addresses this issue by developing a web-based deepfake detection system using Python.

The proposed system effectively analyzes uploaded images and videos using deep learning techniques to determine whether the content is real or fake. By integrating the detection model into a web application, the system provides a user-friendly, accessible, and automated solution for deepfake verification. The system minimizes human intervention, improves detection accuracy, and delivers faster results compared to traditional methods.

This project demonstrates the practical implementation of artificial intelligence in the field of digital forensics and cyber security. It also highlights the importance of developing reliable tools to combat the growing threat of deepfake content. Overall, the system serves as a useful platform for educational, research, and real-world applications, contributing toward enhancing trust and security in digital media.

APENDIX 1

SOURCE CODE

```
From flask import Flask, render_template, request
Import os
Import cv2
Import numpy as np
From tensorflow.keras.models import load_model

App = Flask(__name__)

# Load trained deepfake detection model
Model = load_model("deepfake_model.h5")

UPLOAD_FOLDER = "static/uploads"
App.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

# Image preprocessing function
Def preprocess_image(path):
    Image = cv2.imread(path)
    Image = cv2.resize(image, (224, 224))
    Image = image / 255.0
    Image = np.reshape(image, (1, 224, 224, 3))
    Return image

@app.route("/", methods=["GET", "POST"])
Def index():
```

```
Result = ""
```

```
If request.method == "POST":
```

```
    File = request.files["file"]
```

```
        If file:
```

```
Filepath = os.path.join(app.config["UPLOAD_FOLDER"], file.filename)
```

```
File.save(filepath)
```

```
    Image = preprocess_image(filepath)
```

```
    Prediction = model.predict(image)
```

```
    If prediction[0][0] > 0.5:
```

```
        Result = "Fake"
```

```
    Else:
```

```
        Result = "Real"
```

```
Return render_template("index.html", output=result)
```

```
If __name__ == "__main__":
```

```
App.run(debug=True)
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Deepfake Detection</title>
```

```
<style>
```

```
    Body {
```

```
        Font-family: Arial;
```

```
        Text-align: center;
```

```

        Background-color: #f4f4f4;
    }
    .box {
        Margin-top: 100px;
        Background: white;
        Padding: 30px;
        Display: inline-block;
        Border-radius: 10px;
    }
</style>
</head>
<body>

<div class="box">
<h2>Web-Based Deepfake Detection System</h2>

<form method="POST" enctype="multipart/form-data">
<input type="file" name="file" required><br><br>
<input type="submit" value="Detect">
</form>

    {% if output %}
<h3>Detection Result: {{ output }}</h3>
    {% endif %}
</div>

</body>
</html>

```

REFERENCES

- [1] Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, Montreal, QC, Canada, Dec. 2014, pp. 2672–2680.
- [2] Radford, L. Metz, and S. Chintala, “Unsupervised representation learning with deep convolutional generative adversarial networks,” in *Int. Conf. Learn. Representations (ICLR)*, San Juan, PR, USA, May 2016.
- [3] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein generative adversarial networks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Sydney, NSW, Australia, Aug. 2017, pp. 214–223.
- [4] Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved training of Wasserstein GANs,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, Long Beach, CA, USA, Dec. 2017, pp. 5767–5777.
- [5] T. Karras, T. Aila, S. Laine, and J. Lehtinen, “Progressive growing of GANs for improved quality, stability, and variation,” in *Int. Conf. Learn. Representations (ICLR)*, Vancouver, BC, Canada, Apr.–May 2018.
- [6] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of StyleGAN,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 8107–8116.

- [7] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, and J. Lehtinen, “Alias-free generative adversarial networks (StyleGAN3),” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2021, pp. 12502–12514.
- [8] Brock, J. Donahue, and K. Simonyan, “Large scale GAN training for high fidelity natural image synthesis (BigGAN),” in *Proc. Int. Conf. Learn. Representations (ICLR)*, New Orleans, LA, USA, May 2019.
- [9] P. Isola, J. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 5967–5976.
- [10] J. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks (CycleGAN),” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 2242–2251.
- [11] H. Zhang, T. Xu, H. Li, S. Zhang, X. Huang, and D. Metaxas, “StackGAN: Text to photo-realistic image synthesis with stacked generative adversarial networks,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 5907–5915.
- [12].A.Rössler, D.Cozzolino, L.Verdoliva, C.Riess, J.Thies and M.Nießner, “FaceForensics: A large-scale video dataset for forgery detection in human faces,” 2018, arXiv:1803.09179.
- [13].T. Karras, S. Laine, and T. Aila, “A style-based generator architecture for generative adversarial networks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, Jun. 2019, pp. 4396–4405,

doi: 10.1109/CVPR.2019.00453.

[14].D. Budgen and P. Brereton, “Performing systematic literature reviews in software engineering,” in Proc. 28th Int. Conf. Softw. Eng., New York, NY, USA, May 2006, pp. 1051–1052, doi: 10.1145/1134285.1134500.